Marine Vessel 3D Asset Pipeline: Plan and Tech Stack

Oct 15, 2026 · @Douglas Xu

Summary

USD is the authored source of truth, glTF is a generated delivery format, and the backend owns live data. All three join on one permanent ID per part. Every tool recommended here is open source and runs in containers, so the same stack deploys on a laptop, on-prem or any cloud.

Key decisions:

- **OCX is useful but incomplete for this scope.** Adopt it for hull structure only. For everything else (machinery, piping, outfitting) use a breakdown and ID scheme plus STEP as the CAD exchange format. Details in the next section.
- **Authoring is code-first.** Python generators (build123d or CadQuery) make dimension-true parts and write USD, GLB and STEP from one parameter set. Blender handles hull surfaces, decimation and render checks. FreeCAD is for inspecting STEP and one-off manual parts.
- **CAD is simulated, not skipped.** The generators emit STEP with names and attributes. A separate ingest step reads that STEP into USD exactly as it would read real CAD later, so the integration path is exercised from day one.
- **Web delivery is glTF, not USD.** three.js has only partial USD support, so a build step converts each USD asset to optimized GLB with the stable IDs and properties preserved.
- **Explode and collapse are data.** Each assembly stores an explode vector and distance per child. The viewer computes the offsets; no extra geometry is baked.

OCX assessment and complementary standards

OCX covers hull structure well and little else, so use it for the structural subtree and pair it with other open standards for the rest. The Open Class 3D Exchange schema is an XML schema from the 3DOCX consortium, published under Apache 2.0, with a published 3.0.x

line and Python tooling (the `ocx` package and an OCX validator CLI). Its scope below comes from my background knowledge of the standard, not from those pages; confirm it against the current schema documentation before committing.

Need	OCX 3.x	Gap and recommendation
Hull structure: plates, stiffeners, brackets, panels, frames	Strong fit, designed for class-approval exchange	Use OCX as the input for the structural subtree
Machinery, propulsion, piping, electrical, HVAC, outfitting	Not its purpose	Use STEP AP242 geometry plus your own USD schema for properties
Breakdown and stable IDs	Has its own object identifiers, not a system breakdown	Adopt DNV's open Vessel Information Structure (VIS/Gmod) codes, or SFI as the fallback
Live and sensor data naming	None	ISO 19848 standard data names, mapped to VIS codes
Explode, LOD, web weight	None; XML is verbose	Handled by the USD and glTF layers in this plan
Hull form surfaces	Partial	Keep source surfaces (STEP or NURBS) and tessellate on build

Recommended standards stack:

1. **Structure:** OCX 3.x, parsed with the `ocx` Python package and turned into USD prims (one prim per plate, stiffener or bracket, kept at L1 or L2 detail).
2. **Breakdown and IDs:** VIS/Gmod codes as the human-readable ID, plus a UUID per prim as the permanent key. DNV maintains an open-source VIS SDK (Vista); verify its current license and version before adopting.
3. **Geometry exchange for everything else:** STEP AP242, read through OpenCascade. It is the realistic format a real CAD system will hand you.
4. **Ship-specific STEP protocols (AP215, AP216, AP218):** reference only. They are ISO-paid standards with thin open tooling, so they are not worth building on.

5. **Digital-twin side (optional later):** Asset Administration Shell submodels if you need a standard envelope for equipment data; Eclipse BaSyx is the open implementation.

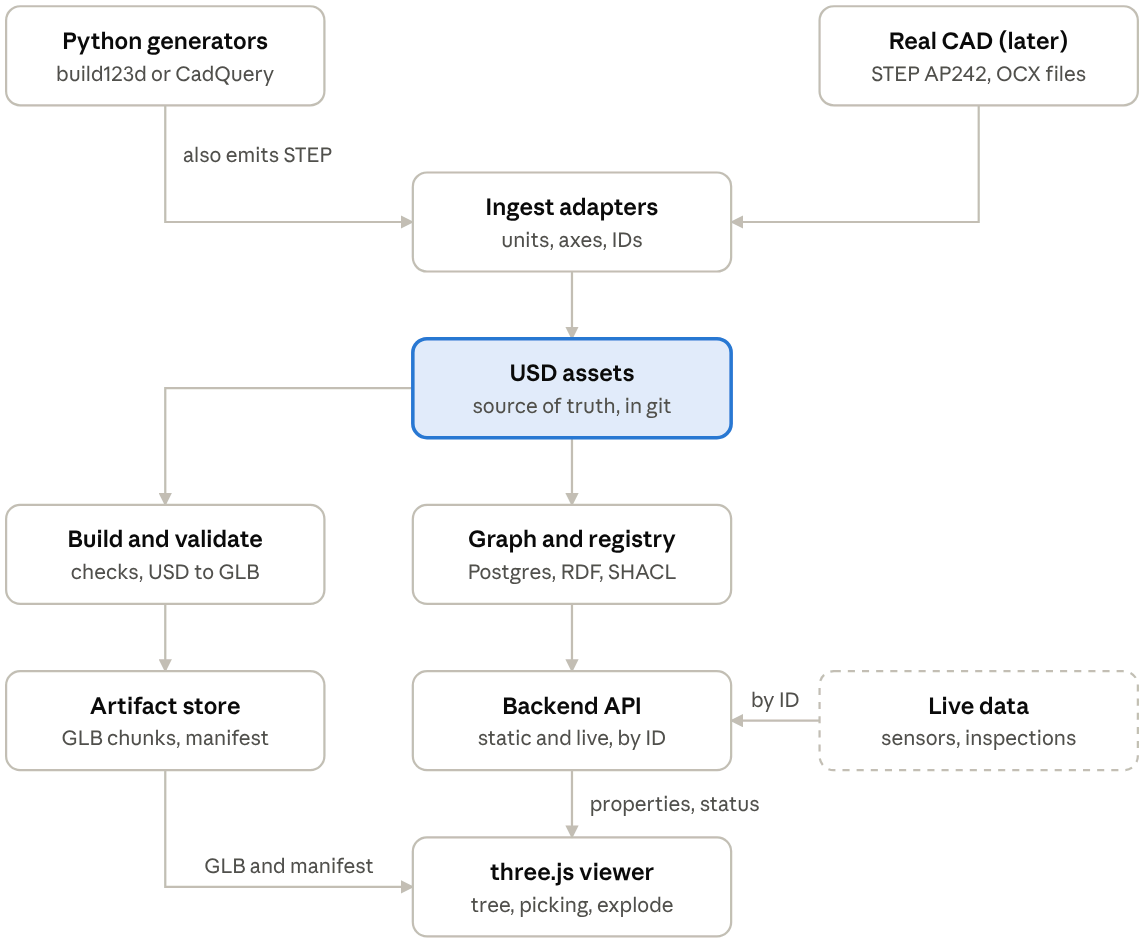
If OCX 3.x turns out to cover more than I have listed, the structural adapter simply grows. Nothing else in the plan depends on its limits.

Architecture

USD sits in the middle: everything upstream feeds it, and everything downstream is rebuilt from it.

One permanent ID joins USD, build outputs, backend and viewer

Accent = source of truth. Dashed = live data kept outside the files. Everything else rebuilds from USD.



system architecture · 10 components

Only the generators (and later real CAD) are edited by people. A click in the viewer traces back to its source part by ID.

Asset model (USD)

Every physical thing is a USD prim at one of five levels, carries a permanent ID, and gets its location from its transform rather than a duplicated attribute.

Hierarchy: Vessel, then Zone or Deck, then System, then Equipment, then Component. Use USD `kind` to mark each level (`assembly` for vessel, zone and system; `component` for equipment; `subcomponent` for parts). That lets any USD tool select and collapse at the right level.

Coordinates: set `metersPerUnit = 1` and `upAxis = Z`. Use the ship frame: origin at aft perpendicular, centerline and baseline; x positive forward, y positive to port, z up. The viewer applies the Y-up conversion once at its root, never per asset.

Properties: a small custom API schema, namespaced `marine:`, applied to each prim. Start as a *codeless* schema (plain USD attributes plus a schema definition file) so any USD-aware tool can read it without a compiled plugin.

Attribute	Type	Purpose
`marine:id`	string (UUID)	Permanent key; the join to backend and ontology. Never reused
`marine:code`	string	Human-readable breakdown code (VIS/Gmod path)
`marine:class`	token	Ontology type, e.g. Propeller, Shaft, Pump, Plate
`marine:function`	string	What it does in the system (e.g. thrust transmission)
`marine:material`	token	Controlled vocabulary entry, with grade and density lookup
`marine:mass`	double (kg)	Declared mass; cross-checked against volume times density
`marine:diameter`, `length`, `thickness`	double (m)	Principal dimensions; the generator's parameters, so they cannot disagree with the mesh
`marine:explodeDir`, `explodeDist`	vector3d, double	Explode behaviour (next section)

Attribute	Type	Purpose
`marine:lodLevel`	int	0 proxy, 1 equipment, 2 full detail

Connectivity: use USD relationships for things that connect without being nested, such as `marine:connectedTo` (a shaft coupling to a gearbox flange) or `marine:serves` (a pump serving a system). The build step mirrors these into the graph store.

Reuse and configuration:

- A shared equipment library of reusable assets, brought into vessels as USD references.
- Variants for configurations, such as fixed-pitch versus controllable-pitch propeller or four versus five blades.
- Payloads on heavy subtrees so a viewer or tool can load them only when asked.
- `PointInstancer` for bolts, rivets and other repeated fasteners, which keeps a flange with 24 bolts to one mesh and 24 transforms.

Explode and collapse

Store the assembled pose as the truth and compute the exploded pose at runtime from each child's explode vector and distance, one hierarchy level at a time.

1. **Assembled transform is canonical.** Children sit at their real positions. Nothing in the file is ever authored in an exploded state.
2. **Each child carries `explodeDir` and `explodeDist`,** relative to its parent's frame. Exploded position = assembled position + `explodeDir` × `explodeDist` × slider value (0 to 1).
3. **One level per step.** Exploding a propulsion system reveals shaft line, gearbox, engine, propeller as siblings. Exploding the propeller then reveals hub, blades, cone, bolts. Offsets accumulate down the tree, so parents carry their children with them.
4. **Collapsed state shows a proxy.** Each assembly has an L0 proxy mesh (a simplified hull of the whole system). Collapsed means proxy visible, children hidden. Expanded means proxy hidden, children visible. This is the logical collapse the viewer needs, and it keeps draw calls low.
5. **Defaults are generated, overrides are authored.** A script assigns default directions by pattern (axial for shaft-line parts, radial for rotationally symmetric parts, vertical for deck-mounted equipment) and spaces them by bounding-box size to avoid overlap. Engineers override only the ones that look wrong.

6. **Validation:** CI checks that exploded siblings never overlap each other's bounding boxes at slider = 1 and that every assembly with children has a proxy.

The same data drives collapse in the model tree panel, so the 3D view and the tree always agree.

Lightweight delivery

Keep three detail levels per asset, enforce a triangle budget per level in CI, and ship one optimized GLB per system with a JSON manifest, so the viewer streams detail only when a user opens a system.

Level	Contents	Initial triangle budget (tune after the spike)
L0	One proxy mesh per assembly	500 to 5,000
L1	Equipment as single meshes, no fasteners	5,000 to 30,000 per system
L2	Full components, fasteners instanced	30,000 to 150,000 per system

The budgets are starting guesses, not measured limits. Set the real numbers by profiling on your slowest target device in the spike.

Build step, run in CI for every changed asset:

- **USD to GLB:** a small Python converter on `pxr` that walks the stage and writes glTF with `pygltflib`. Node names are the stable IDs; `marine:` attributes go into glTF `extras`. A custom converter is preferred over a generic one because explode data, LOD and proxies need custom handling.
- **Optimize:** `glTF-Transform` for dedup and instancing (`EXT_mesh_gpu_instancing`), `meshoptimizer` compression (or Draco), and KTX2 textures. Prefer flat materials from the vocabulary over textures wherever possible; most marine equipment reads fine with colour and roughness alone.
- **Chunk:** one GLB per system or subtree, plus a manifest listing each chunk's ID, URL, bounding box, LOD tier and byte size. The viewer loads hull and exterior first, then streams systems on demand.
- **Hull:** tessellate from the source surfaces at a chord tolerance (coarser for exterior views, finer for close inspection) and keep the original surfaces in the repo.

- **Content hashes** in file names, so browsers and CDNs cache forever and only changed chunks re-download.

USD loading in browsers is still partial, so treat GLB as the web format and USD as the source. Skip converting USD in the browser.

Authoring pipeline: Blender, FreeCAD and Claude

Blender or FreeCAD driven by Claude can work, but the fastest reliable path is a code-first CAD library (build123d or CadQuery) as the main generator, with Blender and FreeCAD in supporting roles. Claude writes and revises Python well, and a script in git is reviewable, repeatable and runs headless in CI; a GUI session is none of those.

Tool	Best use here	Weak spot
build123d or CadQuery (OpenCascade, pure Python)	Dimension-true parametric parts: shafts, flanges, pipes, pumps, propellers, structural members. Parameters drive geometry and `marine:` properties together. Native STEP output	No sculpting or organic surfacing; hull lines need lofting code or imported surfaces
Blender (headless `bpy`)	Hull surface cleanup, decimation, proxy generation, material look, headless preview renders for visual regression	Mesh-based, so dimensions are only as accurate as the tessellation; units and custom-property export need care; not a source of engineering truth
FreeCAD (headless `FreeCADCmd`)	Opening and inspecting STEP, manual one-off parts, acting as a stand-in real CAD when testing ingest	Python API is verbose and document-state heavy; harder for Claude to get right unattended than build123d

Recommended split:

1. **Parametric generators** make most of the assets. Each generator takes a typed parameter object, validates it, builds the solid, tessellates it at set tolerances, and emits USD, GLB and STEP from the same in-memory model, so the three can never drift apart.
2. **Hull and bespoke geometry** come in as NURBS or STEP surfaces (or from Blender for early stand-ins) and pass through a normalization step: units, up axis, naming, ID assignment, material mapping.
3. **Blender runs unattended** to generate L0 proxies and decimated L1 meshes, and to render the preview images used in CI baselines.
4. **Claude's role:** write and extend generators from a short spec, produce the unit tests that check dimensions against parameters, and fix validator failures. A human reviews each generator once; after that, assets are rebuilt from parameters.

A caution and a rule of thumb. Blender's USD export of custom properties as attributes depends on version and settings, so test it on the exact release you pin before relying on it. Build a part as a generator when it comes in standard sizes (pipe schedules, flange classes) or has a diameter in its spec; hand-model it only when it is one-off or organic.

Before committing, run a one-week bake-off: build the same three assets (a controllable-pitch propeller, a centrifugal pump, a hull midship section) with build123d, Blender via Claude, and FreeCAD via Claude. Judge on time to a validated asset, dimensional accuracy, and how much Claude's output needed fixing.

Validation, CI and maintenance

Every change builds in a pinned container and must pass the same validators before an asset is published; nothing reaches the registry by hand.

Repository: one monorepo with `generators/`, `assets/` (USD sources), `library/` (shared equipment), `schema/` (the `marine:` definition and vocabularies), `build/` (converters) and `viewer/`. Large files go through Git LFS. Each asset has a semantic version, and the schema has its own, with migration scripts when it changes.

Validators, run in CI:

Check	What it catches
`usdchecker`	Invalid or non-portable USD
Required properties	Missing ID, class, material, function or dimensions

Check	What it catches
ID uniqueness and stability	Duplicate IDs, or an ID that changed between versions
Units and axis	Wrong `metersPerUnit`, wrong up axis, unapplied scale
Bounding box vs parameters	A mesh that disagrees with its declared diameter or length
Mass check	Declared mass versus volume times material density, within tolerance
Vocabulary	Material or class not in the controlled lists
Ontology conformance	SHACL shapes over the extracted graph: every Pump serves a System, every System has a function
Explode and proxy	Overlaps at slider = 1, assemblies without proxies
Triangle budget	Any LOD tier over its limit
Visual regression	Headless three.js render differs from baseline beyond a threshold

Registry: a Postgres table keyed by asset ID holding version, status (draft, validated, released, deprecated), content hashes of every artifact, the generator and parameter set that produced it, and the toolchain version. A released version is immutable; a fix is a new version.

Reliability habits: pin every tool version in the container image; make builds deterministic (same inputs, same hashes), which makes caching and diffing trivial; keep a small golden set of assets whose outputs are compared byte-for-byte on each toolchain upgrade; and use USD composition (references, variants) rather than copies, so a fix to a library part propagates to every vessel that uses it.

Backend, ontology and simulated CAD integration

The backend splits data by how fast it changes: engineering data lives in USD and the registry, relationships live in a graph, and live data stays outside the files and is fetched by ID at runtime.

API (REST, OpenAPI-described):

- `GET /vessels/{id}/manifest` returns the chunk list and tree for the viewer.
- `GET /assets/{id}` returns static properties and relationships.
- `GET /assets/{id}/children` returns the next explode level.
- `GET /assets/{id}/status` returns live or inspection data (condition, sensor values, work orders), joined by the same ID.
- `POST /ingest` accepts a STEP or OCX file and starts an ingest job; `GET /jobs/{id}` reports progress.

Ontology: define the classes (Vessel, System, Equipment, Component, Material, Function) and relations (`partOf` , `connectedTo` , `serves` , `madeOf` , `suppliedBy`) once, in OWL or SHACL, in the `schema/` folder. The build step extracts USD prims and relationships into the graph with idempotent upserts keyed by `marine:id` . Because the same IDs appear in USD, GLB, graph and API, a query such as "all pumps serving the cooling system and their materials" returns IDs the viewer can highlight directly.

Simulating real CAD: the assets are generated, but the integration is built as if they were not.

1. **CAD simulator.** The generators also write STEP files with product names, colours and attributes (assembly structure, `marine:id` , material, key dimensions) using OpenCascade's XCAF layer, as a real CAD export would.
2. **Ingest adapters, one per source format.** A STEP adapter reads those files back, normalizes units and axes, maps attributes to `marine:` properties, assigns or matches IDs, and writes USD. An OCX adapter does the same for structural models. These adapters never know the geometry was synthetic.
3. **ID matching on re-ingest.** A changed part keeps its ID when its name or attribute key matches; new parts get new IDs; missing parts are flagged for review, not deleted. This is the behaviour real CAD revisions need.
4. **Round trip.** Start one-way (CAD to USD). If you later need edits back in CAD, the shared ID lets you write a change list rather than regenerate geometry.

When a real CAD system arrives, only the source of the STEP or OCX files changes.

three.js viewer

The viewer is driven entirely by the manifest and the stable IDs, so adding a vessel or system needs no viewer code changes.

- **Loading:** fetch the manifest, load hull and exterior chunks first, then stream a system's GLB when the user expands it or its tree node. Use `GLTFLoader` with the meshopt and KTX2 decoders.
- **Scene graph:** node names are IDs. Keep an ID-to-object map so the tree panel, picking and API results all address the same objects.
- **Explode and collapse:** a slider per hierarchy level that animates each child from its assembled position along `explodeDir` by `explodeDist`; collapse swaps the children for the L0 proxy.
- **Picking:** raycast to the nearest ID, walk up to the nearest assembly or component as the user's selection level requires, then call the API and show properties and live status in a side panel.
- **Tools:** section planes, isolate and hide, colour by status or material or system, measurement between two picked points using real metres.
- **Performance:** frustum culling, instancing for fasteners, `BatchedMesh` for large groups of static parts, a visibility budget that drops to L0 for distant systems, and on-demand rendering so the GPU idles when nothing moves.
- **Framework:** plain TypeScript and Vite with a thin UI layer (Svelte or React). Keep the viewer as an embeddable package so it can sit in other products.

Pick a hosting model once: internal engineers can load larger chunks over a fast network; external clients need the lower triangle budgets and a CDN.

Recommended open-source tech stack

Python for everything that builds assets, TypeScript for everything the browser runs, and containers for everything that deploys. Licences below are from memory; check each before adopting.

Layer	Choice	Why	Alternative
Scene and schema	OpenUSD via the `usd-core` Python wheel	Source of truth, composition, variants, relationships	None needed
Parametric CAD	build123d (or CadQuery) on OpenCascade	Code-first, STEP output, easy for Claude to write	FreeCAD scripting

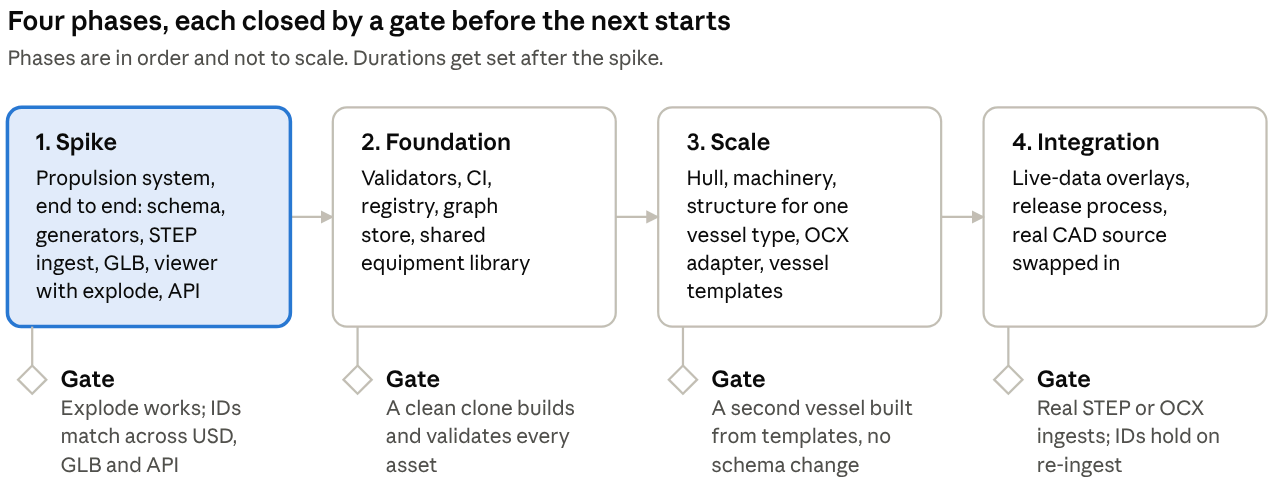
Layer	Choice	Why	Alternative
STEP and XCAF I/O	OCP (OpenCascade Python bindings) or pythonocc	Reads and writes names, colours, attributes	cascadio for quick STEP to GLB
Mesh tools	trimesh, numpy, pygltflib	Tessellation checks, GLB writing	Open3D
Structural input	`ocx` Python package (OCX 3.x)	Typed OCX parsing and validation	Own XML parser
Hull and visual QA	Blender, headless	Decimation, proxies, preview renders	MeshLab
Web optimization	glTF-Transform, meshoptimizer, KTX-Software	Dedup, instancing, compression	gltfpack
Viewer	three.js, TypeScript, Vite	Mature, MIT, large ecosystem	Babylon.js
API	FastAPI with Pydantic, OpenAPI docs	Typed, async, shares Python code with the build	Litestar
Relational store	PostgreSQL with `ltree` and JSONB	Registry, hierarchy queries, flexible properties	SQLite for the spike
Graph and ontology	rdflib and pySHACL for validation; Oxigraph or Apache Jena Fuseki as the store	Standards-based, no lock-in	Postgres with Apache AGE, or Memgraph
Artifact storage	S3-compatible: Garage or SeaweedFS (MinIO is AGPL)	Content-addressed GLB and USD	Plain filesystem for the spike
Orchestration	Dagster, or Make with `just` for a simpler start	Asset-aware pipeline, lineage	Prefect, Snakemake
CI	Forgejo or Gitea Actions, Woodpecker, or GitHub Actions	Runs the same container images	GitLab CI

Layer	Choice	Why	Alternative
Versioning	Git plus Git LFS	Simple and widely supported	DVC
Deployment	Docker Compose first, Helm charts for Kubernetes later	One image set, laptop to cloud	Nomad
Observability	OpenTelemetry, Prometheus, Grafana	Build times, failures, API latency	None needed early

Flexible deployment: build every component as an OCI container, configure through environment variables, keep artifact storage behind an S3-compatible interface and the database behind SQL, and avoid any managed-cloud-only service. That keeps on-prem, air-gapped and cloud deployments interchangeable.

Roadmap, risks and open questions

Start with a spike on one system, and let its results set the budgets, schedule and scope of everything after it.



roadmap · 4 phases, 4 gates

Rough size guesses, to be reset after the spike: the spike about three weeks, the foundation phase four to six weeks. Later phases depend on what the spike shows.

Risks and mitigations

- **OCX scope is assumed from memory.** Read the current schema documentation in the first week and adjust the structural adapter's boundary.
- **Custom USD schemas are not portable by default.** Compiled schema plugins are not loaded by other tools, so start codeless and test the assets in at least one third-party USD viewer.
- **The USD to GLB converter is your own code.** Cover it with round-trip tests (property counts, bounding boxes, ID sets) so fidelity bugs show up in CI.
- **Triangle budgets and chunk sizes are guesses.** Profile on your slowest target device during the spike.
- **Blender's USD export of custom properties varies by version.** Pin the release and test it before relying on it.
- **Licences and standard versions were stated from memory.** Verify each before adoption: VIS and Vista, OCX 3.x, MinIO alternatives, build123d, OpenUSD.

Open questions

- Is there an existing breakdown or ID scheme you must adopt, or is VIS/Gmod acceptable as the default?
- Is the viewer for internal engineers or external clients? That decides triangle budgets, hosting and access control.
- Do you need edits to flow back to CAD, or is USD one-way?
- What are the slowest target devices and browsers?

Sources

Pages seen in search results while preparing this plan; they were not each opened in full.

- OCX schema repository: licence, versioning, working draft status
- OCX package on PyPI: the 3.0.x Python bindings
- OCX tools releases: the validator CLI and schema versions it tracks
- asset-explorer: note that USD support in browser tooling and the three.js Hydra delegate is incomplete